



Least Authority
PRIVACY MATTERS

Ownership Proof Circuit
Security Audit Report

TACEO

Final Audit Report: 2 October 2026

Table of Contents

[Overview](#)

[Background](#)

[Project Dates](#)

[Review Team](#)

[Coverage](#)

[Target Code and Revision](#)

[Supporting Documentation](#)

[Areas of Concern](#)

[Findings](#)

[General Comments](#)

[System Design](#)

[Code Quality](#)

[Documentation and Code Comments](#)

[Scope](#)

[Specific Issues & Suggestions](#)

[Suggestions](#)

[Suggestion 1: Update Stale Prover.toml](#)

[Suggestion 2: Update Incomplete WIP-103 Specification](#)

[Suggestion 3: Bind Proofs to the Intended Operation](#)

[About Least Authority](#)

[Our Methodology](#)

Overview

Background

TACEO has requested that Least Authority perform a security audit of the Ownership Proof circuit, a single Noir circuit built using previously audited Noir building blocks.

Project Dates

- **July 20, 2026 - July 27, 2026:** Initial Code Review (*Completed*)
- **July 28, 2026:** Delivery of Initial Audit Report (*Completed*)
- **October 2, 2026:** Verification Review (*Completed*)
- **October 2, 2026:** Delivery of Final Audit Report (*Completed*)

Review Team

- Mirco Richter, Cryptography Researcher and Engineer
- Burak Atasoy, Project Manager
- Jessy Bissal, Technical Editor

Coverage

Target Code and Revision

For this audit, we performed research, investigation, and review of the Ownership Proof circuit followed by issue reporting, along with mitigation and remediation instructions as outlined in this report.

The following code repository was considered in scope for the review:

- ownership-proof:
<https://github.com/worldcoin/world-id-protocol>

Specifically, we examined the following Git revision for our initial review:

- 6b4642edd348bea9510a92080a3ca324a5a1ecd7

For the verification, we examined the following Git revision:

- 6e8f812a5c5586791352711801e9c40ee8e9da56

For the review, this repository was cloned for use during the audit and for reference in this report:

- world-id-protocol:
<https://github.com/LeastAuthority/world-id-protocol>

All file references in this document use Unix-style paths relative to the project's root directory.

In addition, any dependency and third-party code, unless specifically mentioned as in scope, were considered out of scope for this review.

Supporting Documentation

The following documentation was available to the review team:

- Previous Published Audit Reports:
 - [TACEO - OPRF Noir Circuits](#)
 - [TACEO - OPRF Circom Circuits](#)

In addition, this audit report references the following documents:

- P. Antonino and N. Jain, "Formally Verifying Noir Zero Knowledge Programs with NAVE." *arXiv* 2026, [AJ26]
- P. S. L. M. Barreto and M. Naehrig, "Pairing-Friendly Elliptic Curves of Prime Order." *IACR Cryptology ePrint Archive*, 2005, [BN05]
- D. J. Bernstein, N. Duif, T. Lange, P. Schwabe, and B. Yang, "High-speed high-security signatures." *IACR Cryptology ePrint Archive*, 2011, [BDL+11]
- L. Grassi, D. Khovratovich, C. Rechberger, A. Roy, and M. Schofnegger, "Poseidon: A New Hash Function for Zero-Knowledge Proof Systems." *IACR Cryptology ePrint Archive*, 2019, [GKR+19]
- L. Grassi, D. Khovratovich, and M. Schofnegger, "Poseidon2: A Faster Version of the Poseidon Hash Function." *IACR Cryptology ePrint Archive*, 2023, [GKS23]
- W. Kneal, "Solvable Cases of the Decision Problem, by W. Ackermann," *Philosophy*, vol. 31, no. 116, pp. 92–93, 1956, [Kneal56].
- A. Ozdemir, G. Kremer, C. Tinelli, and C. Barrett, "Satisfiability Modulo Finite Fields." *IACR Cryptology ePrint Archive*, 2023, [OKT+23]
- NAVE:
<https://github.com/blockhouse-sec/nave>
- Noir compiler fork:
<https://github.com/blockhouse-sec/noir>
- Noir/ACIR documentation:
<https://noir-lang.org/docs>
- Noir / nargo:
<https://noir-lang.org>
- cvc5 1.3.4 (GPL build, CoCoALib-enabled):
<https://github.com/cvc5/cvc5>
- WIP-103: Proof of Ownership:
<https://github.com/LeastAuthority/world-id-protocol/blob/main/docs/WIPs/wip-103.md>
- RFC 2119 | Key words for use in RFCs:
<https://datatracker.ietf.org/doc/html/rfc2119>
- EIP-2494:
<https://eips.ethereum.org/EIPS/eip-2494>
- TaceoLabs noir-poseidon:
<https://github.com/TaceoLabs/noir-poseidon>
- TaceoLabs oprf-service:
<https://github.com/TaceoLabs/oprf-service>
- iden3 circomlib:
<https://github.com/iden3/circomlib>
- HorizenLabs poseidon2:
<https://github.com/HorizenLabs/poseidon2>

Areas of Concern

Our investigation focused on the following areas:

- Correctness of the implementation;
- Vulnerabilities within each component and whether the interaction between the components is secure;
- Protection against malicious attacks and other ways to exploit;
- Inappropriate permissions and excess authority;
- Data privacy, data leaking, and information integrity; and
- Anything else as identified during the initial analysis phase.

Findings

General Comments

The World ID Protocol codebase implements a privacy-preserving system that allows individuals to prove they are unique humans without revealing their identity.

This audit covers the Noir circuit in `crates/proof/noir/ownership-proof`, which implements [WIP-103](#). The circuit proves that a user controls one of up to seven EdDSA public keys committed to a leaf in the World ID Registry without revealing the selected key, leaf index, Merkle path, or commitment blinder. It verifies an EdDSA signature over a nonce-bound Poseidon2 message, reconstructs the registry root from the private key set and Merkle proof, and publicly outputs a domain-separated commitment to the hidden leaf index and blinder.

Our team manually reviewed the code within the defined scope. This included checking the commitment, signed-message, and Merkle-leaf domain-separator constants against their specified byte-string encodings, and mapping each of the seven normative validations of WIP-103 to its implementation in the circuit. The review excluded the prover, the verifier, and nonce system, as discussed in the [Scope](#) section. We reviewed the commitment and message constructions, the Merkle-path computation, including its branchless selection logic and index arithmetic, the key-array lookup, the range checks and field-to-integer casts, and the EdDSA verification with its BabyJubJub dependency, several elements of which were previously audited as part of an [earlier review](#). We also executed the circuit against the repository's example inputs under the project's pinned toolchain, reviewed the dependency pins, and analyzed the replay surface across use cases and operations. We found that the checked-in example inputs no longer satisfy the circuit and appear to predate the WIP-103 message rework ([Suggestion 1](#)). Our review identified normative constraints and security-critical verifier obligations that the implementation enforces or presumes but that the specification does not state, making a compliant reimplementing potentially insecure ([Suggestion 2](#)). In addition, we identified that proofs are not cryptographically bound to the operation the user intended, so protection against first-use substitution between operations within the same use case rests entirely on issuer-side nonce discipline, which lies outside the audited scope ([Suggestion 3](#)).

As a separate internal experiment, we used AI together with [NAVe](#) to compile the WIP-103 Ownership Proof circuit to ACIR and translate it into SMT-LIB. We discharged verification conditions using cvc5's finite-field solver [\[OKT+23\]](#), supplemented by a structural checker we developed for statements beyond the solver's practical reach, including hash-cone determinism and functional-congruence claims, and by an uninterpreted-function abstraction of the cryptographic primitives for circuit-level statements. Within the resulting model, we proved, for all admissible inputs, the soundness of several aspects of the circuit's structural logic: the Merkle-path selection and index arithmetic, the range and key-index bounds, the unique determination of every unconstrained hint in the elliptic-curve gadgets, and the correctness of the

Merkle-root computation against its textbook specification for every depth and index position accepted by the circuit. With the hash and signature primitives abstracted as pure functions, we further proved that the circuit's public output, signed message, and acceptance condition are wired exactly as required by the specification (see [here](#) for additional details).

All of these results depend on assumptions connecting the model to the original circuit and its toolchain: the translation pipeline, mechanically patched dependency sources, abstraction axioms, and cryptographic properties of the primitives themselves, which inherently fall outside the scope of such tools. We converted those assumptions into audit checks (see [here](#)) and applied them to identify areas requiring additional manual review. However, because this portion was generated primarily using AI, the resulting proofs should be considered experimental.

System Design

Our team reviewed the Ownership Proof circuit and found that it has been well designed, with security at the forefront of its architecture. Although proving key ownership by enforcing $pk = sk \cdot G$ would be simpler and likely less expensive at the circuit level, doing so would require exposing the long-lived secret key to the zero-knowledge prover as a private witness. Instead, the circuit verifies an externally generated signature over the commitment and nonce, potentially allowing the key to remain isolated within a secure signer while supporting per-request policy and user-consent checks.

Code Quality

We performed a manual review of the repositories in scope and found the code to be of high quality, well organized, and aligned with development best practices. The implementation adapts mature circuits that have undergone prior audits, such as the [OPRF Noir Circuits](#) audit, and we did not identify any security-relevant issues.

Tests

Test coverage of the Ownership Proof Noir circuit is limited. Ten unit tests cover only the commitment and Merkle-root computation, including depths 1–2 of 30 and only some index positions. The two negative tests cover only Merkle bounds. The `main` function has no circuit-level tests. Key selection, the `pk_index` bound, EdDSA verification, and the nonce constraint are untested, and the end-to-end fixture is stale. We estimate that the tests cover less than half of the happy path and only a small fraction of the security-relevant rejection paths. Notably, rejection of an invalid signature is never tested. We recommend adding negative tests for the signature and key-selection asserts, along with a valid `main` fixture.

Documentation and Code Comments

The project documentation provided for this security review was insufficient, as the specification is incomplete. We recommend improving the project documentation ([Suggestion 2](#)).

Scope

The audit covered only the Ownership Proof circuit. The prover, verifier, nonce system, and consuming system, all of which are critical to preventing certain attacks, were excluded. We therefore consider the scope insufficient to assess the security of the Ownership Proof System as a whole because several security-critical obligations exist outside the circuit and are not stated normatively in the specification ([Suggestion 2](#)). Because a sound circuit with a verifier that omits any of these checks or a consumer that does not handle the requested operations properly yields an insecure system, we recommend a follow-up audit of the consumer, prover, and verifier.

Specific Issues & Suggestions

We list the issues and suggestions identified during the review in descending order of severity. In most cases, remediation of an issue is preferable, but mitigation is suggested as another option for cases where a trade-off could be required.

ISSUE / SUGGESTION	SEVERITY	STATUS
Suggestion 1: Update Stale Prover.toml	Informational	Resolved
Suggestion 2: Update Incomplete WIP-103 Specification	Informational	Resolved
Suggestion 3: Bind Proofs to the Intended Operation	Informational	Resolved

Suggestions

Suggestion 1: Update Stale Prover.toml

Location

[crates/proof/noir/ownership-proof/Prover.toml](#)

Synopsis

The checked-in example inputs do not satisfy the circuit: execution with the pinned toolchain fails at `assert(is_valid, "Invalid signature")` in `main.nr`. The header still reads "OPRF Nullifier Circuit," and the fixture predates the WIP-103 message rework, so its signature no longer matches the signed message computed by the circuit.

Mitigation

We recommend regenerating the fixture with the current message construction or removing it.

Status

The TACEO team has replaced the stale fixture in PR 885, and all 42 Noir tests and direct fixture execution now pass.

Verification

Resolved.

Suggestion 2: Update Incomplete WIP-103 Specification

Location

[docs/WIPs/wip-103.md](#)

Synopsis

The specification omits normative constraints and verifier obligations on which the security of the Ownership Proof depends. Several of these requirements are present in the implementation but not required by the specification, so a compliant reimplementaion could be insecure. Specifically:

1. Leaf-index binding: The specification never requires the `leafIndex` hashed into the commitment in steps 1 and 7 to equal the position proven by the Merkle path in step 6, such as through bit

decomposition into the path direction bits, which also bounds it to 2^{depth} . Without this binding, control of any leaf, combined with another user's `leafIndex` and blinder, allows impersonation. The implementation correctly binds both through a shared witness.

2. Key-list and key validation: The encoding, arity, and padding of $H(pk[\])$ are undefined. The specification does not require the proven `pk` to be a prime-order-subgroup point, distinct from the identity, low-order points, or padding values. It also does not require canonical EdDSA checks, including requiring `S` to be below the subgroup order and `R` to be on-curve.
3. Variable depth: Variable depth is a public input with undefined circuit-side semantics. The specification defines no fixed depth per verifying key, padding scheme, or domain separation between leaf and internal nodes, creating a second-preimage risk.
4. Operation binding: The signed message $H(\text{tag}, \text{expected_commitment}, \text{nonce})$ is identical for reissuance, status, and deletion. A relayed proof can therefore authorize an operation other than the one intended.
5. Root freshness: "A root hash for a valid Merkle tree" leaves the acceptance of historical roots undefined. Proofs using pre-rotation keys associated with an old root must not verify because reissuance after loss is specifically a key-rotation case.
6. Nonce: The entropy, size (which must fit within a field element), expiry, and uniqueness scope are unspecified.
7. Private inputs: Private inputs, including the witness, are not enumerated, and the proving system, curve, and field are absent from the normative section.
8. Leaf format: `offchainSignerCommitment` and the leaf format are undefined because "inner value of the leaf index" is ambiguous.
9. Issuance-time precondition: The proof is meaningful only if `sub` was verifiably bound to the user's leaf at enrollment, and account recovery preserves `leafIndex`. Otherwise, reissuance fails.

In addition, step 3 proves possession of a valid signature rather than "knowledge of `sk`." Steps 1 and 7 duplicate each other. RFC 2119 terminology is used inconsistently, for example, "must" versus "IS REQUIRED." Finally, the cross-context replay resistance described in Security §2 results from the verifier matching `expected_commitment` because the private domain separator is not visible to the verifier.

Mitigation

We recommend amending the specification to state the enumerated constraints and verifier obligations normatively and adding the missing definitions, encodings, and test vectors. Until then, the implementation, rather than the specification, should be treated as the authoritative definition, with conformance verified case by case.

Status

The TACEO team has updated the WIP-103 specification in this [PR](#).

Verification

Resolved.

Suggestion 3: Bind Proofs to the Intended Operation

Location

[crates/proof/noir/ownership-proof/src/main.nr](https://github.com/leastauthority/taceo-circuits/blob/main/crates/proof/noir/ownership-proof/src/main.nr)

Synopsis

All operations within a use case, including status, reissuance, renewal, and deletion, share the same commitment and message domain separators. As a result, a proof attests to $\{\text{expected_commitment}, \text{nonce}, \text{root}, \text{depth}\}$, but not to the intended operation. One-time nonce consumption prevents

resubmission but not first-use substitution: a proof generated for a status request verifies equally at the deletion endpoint if submitted to that endpoint first. The specification's claim that proofs cannot be replayed "in another context" holds only across use cases with distinct commitment domains, not between operations within a single use case. Protection therefore depends entirely on issuer-side nonce-store discipline. Because the verifier and nonce system are out of scope, we cannot assess whether this protection is implemented within those components.

Mitigation

We recommend cryptographically binding an operation identifier and, ideally, the issuer's identity into the signed message or public statement while leaving the credential commitment unchanged. This could be achieved, for example, through a context field bound using an inexpensive constraint. At minimum, the specification should require nonces that are operation-scoped, session-scoped, expiring, atomically consumed, and rejected at every endpoint other than the one for which they were issued.

Status

The TACEO team has bound a verifier-supplied context into the signed message and verification inputs, with mismatched contexts now rejected.

Verification

Resolved.

About Least Authority

Founded in 2011 and based in Berlin, Germany, since 2016, Least Authority provides security consulting for privacy-preserving technologies and decentralized systems. We are committed to transparency, openness, and advancing secure digital infrastructure.

We believe that people have a fundamental right to privacy and that secure solutions enable people to use the internet and other connected technologies more freely. Our security consulting services help teams make their solutions more resistant to unauthorized access and unintended system manipulation. We support teams from the design phase through production launch and beyond.

Our team has experience reviewing code for security vulnerabilities and specific attack vectors in multiple languages, including Rust, Go, Python, C, C++, Solidity, Cairo, Circom, Noir, Haskell, OCaml, TypeScript, Swift, Zokrates, and others. We have reviewed implementations of cryptographic protocols and distributed system architecture in blockchains, payments, smart contracts, zero-knowledge protocols, and consensus protocols. Additionally, we use various tools to scan code and networks and build custom tools as necessary.

For more information about our security consulting, please visit:

<https://leastauthority.com/security-consulting>.

Our Methodology

We use a transparent review process that provides the project team visibility into our findings, assumptions, and remediation considerations throughout the audit. The goals of our security audits are to improve the quality of the systems we review and support sufficient remediation to help protect users. The following is the methodology we use in our security audit process.

Code Review

In manually reviewing all of the code, we identify potential issues, including those related to code logic, error handling, cryptographic implementations, and correctness with respect to specifications, standards, and best practices. We also evaluate areas where more defensive programming could reduce the risk of future mistakes and accelerate future audits. Although our primary focus is on the in-scope code, we examine dependency code and behavior when it is relevant to a particular line of investigation.

When appropriate, we also use artificial intelligence (AI) to support the efficiency of our security audits without reducing rigor or reliability. AI-assisted outputs are carefully guided, reviewed, and validated by our team so that conclusions remain accurate and defensible. This supports stronger audit decisions, not only faster review.

Vulnerability Analysis

Our audit techniques include manual code analysis, user interface interaction, and white box penetration testing, supplemented by various tools, including AI, that support the code review process. Discussions with the development team help clarify their vision for the software. Installing and using the relevant software helps us understand user interactions and roles, while also informing our analysis of threat models and attack surfaces. Design documentation, relevant papers, other audit results, similar projects, source code dependencies, and open issue tickets may also be reviewed to inform the analysis. We identify potential vulnerabilities, and, when appropriate, create issue entries. For each entry, we follow the issue investigation and remediation process described below.

Documenting Results

We follow a conservative and transparent process for analyzing potential security vulnerabilities and supporting successful remediation. Whenever a potential issue is identified, we raise it with the client through a dedicated secure communication channel to discuss its validity and impact, to the extent these can be determined at that stage. This process allows potentially relevant findings to be reviewed with the client early, before they are documented in greater detail in the initial audit report. For each confirmed issue, we assess impact and feasibility and use those ratings to determine the overall severity, following the [OWASP Risk Severity Matrix](#).

Suggested Solutions

We identify immediate and comprehensive mitigations available to live deployments and then suggest remediation requirements for future releases. When code is already deployed in production, we include mitigation strategies in the report to provide interim support until appropriate remediation is available. The mitigation and remediation recommendations should be reviewed by the development team and deployment engineers. After delivering the Initial Audit Report, we remain available to support implementation of the proposed mitigations and remediations before the verification review.

Development teams should address findings in an order and with an urgency that prioritizes the security of their users. Findings should be addressed as soon as possible, but remediation should be carried out pragmatically and should not cause unnecessary disruption to project timelines.

Although issues must be handled on a case-by-case basis, we work toward a resolution timeline that balances the impact on users and the needs of the project team.

Resolutions & Publishing

Once the findings are comprehensively addressed, we complete a verification review to assess whether the issues and suggestions have been sufficiently addressed. When this analysis is complete, we update the report and provide a Final Audit Report that can be published in full. If any critical issues remain, we recommend that the Final Audit Report not be published and that users and other stakeholders be alerted to the impact. We encourage all findings to be addressed and the Final Audit Report to be shared publicly to support transparency and advance security knowledge within the industry.